

TECNOLOGÍAS DE LA INFORMACIÓN EN LÍNEA

DESARROLLO DE APLICACIONES WEB

3 créditos

**Profesor:**

Ing. Manuel Bravo, Mg.

Ing. Tatiana Cobeña Macías, Mg.

Ing. Edison Solorzano, MG.

Asignatura	Semestre
DESARROLLO DE APLICACIONES WEB	Sexto

Tutorías: El profesor asignado, se publicará en el entorno virtual de aprendizaje online.utm.edu.ec), y sus horarios de conferencias se indicarán en la sección **CAFETERÍA VIRTUAL**.

PERÍODO ACADÉMICO:

MAYO 2023 / SEPTIEMBRE 2023

Contenido

Unidad III: Arquitectura orientada a microservicios web y manejo de estados.	3
Tema 1: Arquitectura orientada a servicios (SOA).....	3
Tema 2: INTRODUCCIÓN A MICROSERVICIOS	12
Tema 3: FRAMEWORKS	18



Resultado de aprendizaje de la asignatura

Este curso provee a los estudiantes el conocimiento y la experiencia práctica para diseñar e implementar aplicaciones web cumpliendo con los estándares actuales y las buenas prácticas de programación que faciliten su mantenibilidad, escalabilidad y adaptabilidad.



Ilustraciones gráficas



Sabías que. - La presente imagen dentro del manual mostrara información interesante y novedosa de la asignatura.



Recuerde que. - La presente imagen dentro del manual permite recordar información que es relevante y que vas necesitar en tu vida profesional.



Comprueba tu aprendizaje. - Es un cuestionario de un conjunto de preguntas que se confecciona para obtener información con algún objetivo en concreto. Por cada tema de la unidad se tendrá un cuestionario que el estudiante debe resolver entre preguntas teóricas y prácticas.



Videos. - Para complementar contenidos de la unidad dentro del manual se tiene videos que permitirán al estudiante revisar y explorar conocimientos auditivos y visuales.



Curiosidades. - La presente imagen en el manual mostrara información que debes conocer de la asignatura.



Datos útiles. - La presente imagen en el manual mostrara información que deberás tomar en cuenta en otras unidades de la asignatura o de otras asignaturas en semestres superiores.



DESARROLLO DE APLICACIONES WEB



Unidad III: Arquitectura orientada a microservicios web y manejo de estados.

Resultado de aprendizaje de la unidad: Diseñar, desarrollar y mantener servicios y aplicaciones en tecnologías web e integrarlas en los sistemas de información corporativos.



Tema 1: Arquitectura orientada a servicios (SOA)

Muchas empresas han dedicado grandes inversiones en los recursos de sus sistemas software a lo largo de los años. Dichas empresas tienen una enorme cantidad de datos almacenados en sistemas de información (EIS) legacy, de forma que no resulta práctico descartar dichos sistemas existentes. Es mucho más efectivo evolucionar y mejorar los EIS. ¿Pero cómo hacer esto? La arquitectura orientada a servicios (Service Oriented Architecture) proporciona una solución con unos costes aceptables. Si bien la arquitectura a servicios no es nueva, se está convirtiendo en el principal framework para la integración de entornos heterogéneos y complejos. En esta charla, explicaremos con detalle qué es la arquitectura orientada a servicios.

El software de empresa está estrechamente relacionado con su organización interna, con los procesos que la forman, y con el modelo de negocio de dicha empresa. El software de empresa está condicionado tanto por las dependencias entre sus departamentos, como por las relaciones con negocios externos.

En el software de empresa, el arquitecto adquiere el rol de un influenciador y controlador externo. Tiene la responsabilidad de dirigir proyectos software individuales tanto desde el punto de vista estratégico de la organización en su totalidad, como desde el punto de vista táctico (punto de vista de la meta a alcanzar con el proyecto individual). Tiene que equilibrar diferentes requerimientos a la vez que intentar crear un orden perdurable dentro del ámbito del software de empresa. La arquitectura del software de empresa es una de las herramientas más importantes con las que cuenta el arquitecto. Éste tiene que enfrentarse constantemente con cambios y adiciones de funcionalidades que incrementan la complejidad del sistema y reducen su eficiencia. Mediante la refactorización de las soluciones actuales, los arquitectos luchan para intentar reducir la complejidad e incrementar la agilidad del sistema.

CARACTERÍSTICAS DE UNA ARQUITECTURA ORIENTADA A SERVICIOS

- **Simplicidad:** A pesar de la cantidad de gente implicada, todos deben ser capaces de comprender y gestionar la arquitectura en sus niveles respectivos (por ejemplo, especificar nuevas funcionalidades a nivel de negocio, implementarlas y mantenerlas).
- **Flexibilidad y mantenibilidad:** La arquitectura debe definir diferentes componentes que puedan reordenarse y reconfigurarse de una forma flexible. No se debe permitir que cambios locales tengan un efecto sobre el sistema global.
- **Reusabilidad:** Uno de los aspectos más importantes de la reusabilidad es la de compartir datos en tiempo real, así como el compartir funcionalidades comunes.
- **Desacoplamiento entre funcionalidad y tecnología:** La arquitectura debe hacer que la organización de la empresa sea independiente de la tecnología. En particular, la arquitectura debería evitar dependencias de productos y vendedores específicos.

SERVICIO

El término "servicio" se ha venido utilizando desde hace bastante tiempo y de muchas formas diferentes. Hoy, por ejemplo, encontramos grandes compañías, como por ejemplo IBM, que promocionan el concepto de "servicios bajo demanda". Con la llegada del siglo 21, el término "servicios Web" se ha convertido extremadamente popular, si bien ha sido utilizado a menudo para hacer referencia a diferentes conceptos de computación. Por ejemplo, algunos lo utilizan para referirse a servicios de aplicaciones que se ofrecen a los

usuarios a través de la Web, como la aplicación salesforce.com. Otros utilizan el término "servicios Web" como módulos de aplicaciones que son accesibles para otras aplicaciones a través de Internet, mediante protocolos basados en XML. Hay que decir que servicios Web y SOA NO son equivalentes, sin embargo, como veremos más adelante, se pueden utilizar servicios Web para implementar una arquitectura SOA.

Para nosotros, el término servicio hará referencia a alguna actividad significativa que un programa de ordenador realiza o solicita a otro programa de ordenador. O, en términos más técnicos, un servicio es un módulo de aplicación auto contenido que es remotamente accesible. Por ejemplo, los frontends o presentaciones de las aplicaciones proporcionan accesibilidad a los servicios. A veces, los términos "cliente" y "servidor" se utilizan como sinónimos de "consumidor de un servicio" y "proveedor de un servicio", respectivamente.

Además, los servicios proporcionan un nivel de abstracción que oculta muchos detalles técnicos, incluyendo la localización y búsqueda del servicio. Típicamente, los servicios proporcionan funcionalidad de negocio, en lugar de funcionalidades técnicas. Otra característica de los servicios es que no se diseñan para un cliente específico, en su lugar constituyen una facilidad que contribuye a satisfacer alguna demanda pública. Es decir, proporcionan una funcionalidad que puede reutilizarse en diferentes aplicaciones. La "rentabilidad" del servicio dependerá del número de clientes diferentes que utilicen el servicio, o lo que es lo mismo, del nivel de reutilización conseguido.

ELEMENTOS QUE COMPONEN LA ARQUITECTURA SOA

Una arquitectura orientada a servicios está basada en cuatro elementos clave: frontend de la aplicación, servicio, repositorio y bus de servicios.



Si bien el frontend de la aplicación es el propietario del proceso del negocio, los servicios proporcionan la funcionalidad del negocio que los frontends de las aplicaciones y otros servicios pueden utilizar.

Un servicio consiste en: (a) una implementación que proporciona lógica del negocio y datos, (b) un contrato del servicio que especifica la funcionalidad, uso, y restricciones para el cliente (que puede ser un frontend de una aplicación u otro servicio), y (c) una interfaz del servicio que físicamente expone la funcionalidad.

El repositorio de servicios almacena los contratos del servicio de los servicios individuales de una SOA, y el bus de servicios interconecta los frontends de las aplicaciones y los servicios.



Sabías que. - Una arquitectura orientada a servicios (SOA) **es una arquitectura software basada en los conceptos clave de frontend de aplicaciones, servicio, repositorio de servicios, y bus de servicios.** Un servicio está formado por un contrato, uno o más interfaces y una implementación. Los bloques de construcción de SOA son los servicios, los cuales están débilmente acoplados para favorecer su reutilización y son altamente interoperables a través de sus contratos. Los servicios en sí mismos son "ajenos" a las interacciones requeridas a nivel de transporte para hacer posible la comunicación entre el suministrador del servicio y el consumidor del servicio.

FRONTEND DE LAS APLICACIONES

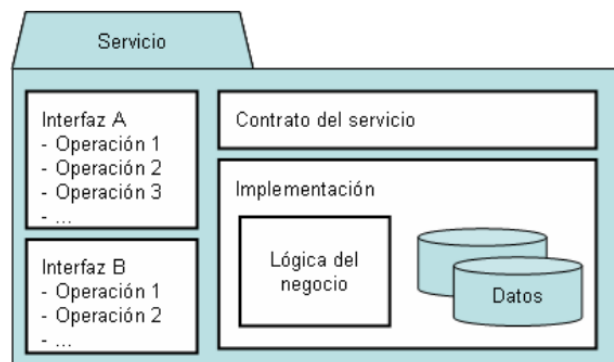


Los frontends de las aplicaciones son los elementos activos de la arquitectura SOA. Éstos inician y controlan todas las actividades de los sistemas corporativos. Hay diferentes tipos de frontends. Un frontend de una aplicación puede presentar una interfaz gráfica de usuario, como por ejemplo una aplicación Web o un cliente rico que interactúa directamente con los usuarios finales. Los frontends no tienen que interactuar

necesariamente de forma directa con los usuarios finales. Los programas por lotes o los procesos de larga duración que invocan a las funcionalidades de forma periódica o como resultado de eventos específicos son también ejemplos válidos de frontends.

SERVICIOS

Un servicio es un componente software con un significado funcional que lo distingue de otros, y que típicamente encapsula un concepto de negocio de alto nivel. A continuación, se muestra los elementos que forman parte de un servicio.



El contrato del servicio proporciona una especificación informal sobre el propósito, funcionalidad, restricciones, y uso del servicio. La forma de dicha especificación puede variar dependiendo del tipo de servicio. La definición formal de la interfaz basada en lenguajes tales como IDL y WSDL no es obligatoria. Si bien añade un beneficio significativo: proporciona una abstracción e independencia de la tecnología, incluyendo al lenguaje de programación, protocolo de red y entorno de ejecución. Es importante comprender que un contrato de un servicio proporciona más información que una especificación formal. El contrato puede imponer una determinada semántica sobre la funcionalidad o el uso de parámetros que no están sujetos a especificaciones IDL o WSDL.

La interfaz expone la funcionalidad del servicio a los clientes que están conectados a dicho servicio a través de la red. Si bien la descripción de la interfaz forma parte del contrato del servicio, la implementación física de la misma consiste en unos stubs de servicio, que se incorporan en los clientes de un servicio.

La implementación del servicio proporciona la lógica del proceso, así como los datos requeridos. Es la realización técnica que satisface el contrato del servicio. La

implementación del servicio consiste en uno o más artefactos tales como programas, datos de configuración y bases de datos.



Recuerde que. - Los servicios son el corazón de una arquitectura SOA y deben ser: débilmente acoplados, de grano grueso (coarse-grained: implementan funcionalidades de alto nivel), centrados en el negocio y reutilizables.

El término **acoplamiento** hace referencia al grado en el que los componentes software dependen unos de otros. El acoplamiento puede tener lugar a diferentes niveles. Los procesos de negocio requieren un alto nivel de flexibilidad, y por lo tanto una arquitectura con un bajo acoplamiento para así poder reducir la complejidad total y las dependencias, y en consecuencia facilitar cambios más rápidos y con menores riesgos.

Nivel	Tight coupling	Loose coupling
Acoplamiento físico	Requiere conexión física directa	Utiliza un intermediario físico
Estilo de comunicación	Síncrono	Asíncrono
Sistema de tipos	Interfaz explícita con nombres de operaciones y argumentos fuertemente tipados	Formato de mensajes flexible
Patrón de interacción	Navegación a través de complejos árboles de objetos	Centrado en los datos, mensajes autocontenidos
Control de la lógica del proceso	Centralizado	Componentes distribuidos lógicamente
Descubrimiento y enlazado de servicios	Estático	Dinámico
Dependencia de la plataforma	Dependencia fuerte del sistema operativo y lenguaje de programación	Independencia del sistema operativo y lenguaje de programación

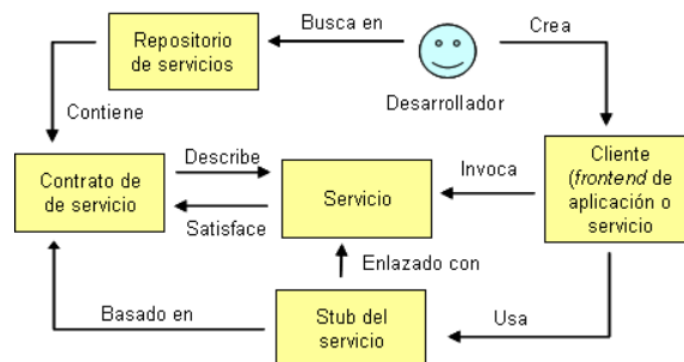
REPOSITORIO DE SERVICIOS

El repositorio de servicios proporciona facilidades para encontrar servicios y adquirir toda la información para utilizar los servicios, particularmente si estos servicios deben encontrarse fuera del ámbito funcional y temporal del proyecto que los creó. Si bien mucha de la información requerida forma parte del contrato del servicio, el repositorio de servicio puede proporcionar información adicional, tal como la localización física, información sobre el proveedor, personas de contacto, tasas de uso, restricciones técnicas, cuestiones sobre seguridad, y niveles de servicio disponibles.

Un repositorio de servicios puede ser arbitrariamente simple; en un extremo, podríamos no requerir usar ninguna tecnología. Un lote de contratos de servicio impresos localizados en una oficina y accesible por todos los proyectos es un repositorio de servicios válido. Sin embargo, hay mejores formas de proporcionar esta información, manteniendo la simplicidad del repositorio, como por ejemplo utilizar alguna base de datos propietaria que contiene datos administrativos y contratos de servicios más o menos formales para cada versión de un servicio.

Es importante distinguir entre enlazado de servicios (binding of services) en tiempo de desarrollo y en tiempo de ejecución. El binding (utilizaremos los términos binding o enlazado indistintamente) hace referencia a la forma en la que las definiciones de los servicios y las instancias de los servicios son encontradas, incorporadas en la aplicación del cliente, y finalmente enlazadas a nivel de red.

En la siguiente figura, se describe un proceso en el que los servicios se encuentran en tiempo de desarrollo. En este caso, el desarrollador es responsable de localizar toda la información requerida en el repositorio de servicios para crear un cliente que interactúe correctamente con la instancia del servicio.



BUS DE SERVICIOS

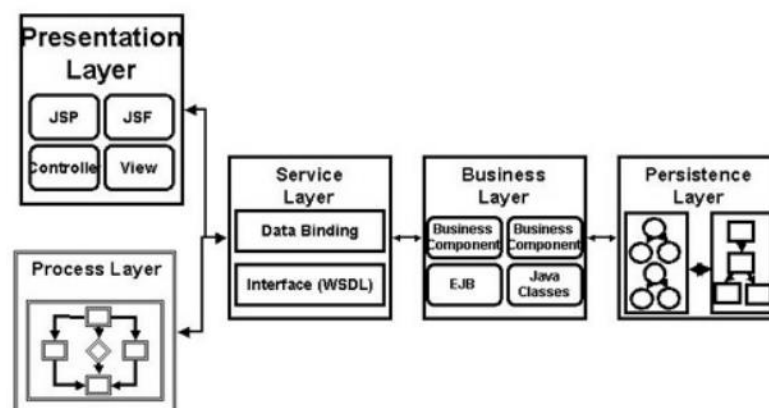
Un bus de servicios (ESB: Enterprise Service Bus) conecta a todos los participantes de una SOA (tanto servicios como frontends). El bus de servicios es similar al concepto de bus software definido en el contexto de CORBA. Aunque existen diferencias significativas, entre ellas la más importante es que el bus de servicios no necesariamente debe estar formado por una única tecnología, sino que puede comprender varios productos y conceptos.

Las principales características de un bus de servicios son:

- **Conectividad:** el objetivo principal de un bus de servicios es del de interconectar a los participantes de una SOA. Por lo tanto, debe proporcionar facilidades para permitir a los participantes de una SOA invocar las funcionalidades de los servicios.
- **Heterogeneidad de tecnología:** puesto que la realidad de las empresas se caracteriza por tecnologías heterogéneas, el bus de servicios debe ser capaz de conectar participantes basados en diferentes lenguajes de programación, sistemas operativos o soporte de ejecución. Además, normalmente encontraremos muchos productos middleware y protocolos de comunicación en la empresa, todas las cuales deben ser soportadas por el bus de servicios.
- **Heterogeneidad de conceptos de comunicación:** es similar a lo expuesto anteriormente, pero aplicado a las comunicaciones. Obviamente, el bus de servicios debe permitir comunicaciones síncronas y asíncronas.
- **Servicios técnicos:** si bien el propósito del bus de servicios es fundamentalmente la comunicación, también debe proporcionar servicios técnicos tales como logging, auditorías, seguridad, transformación de mensajes o transacciones.

CAPAS EN APLICACIONES ORIENTADAS A SERVICIOS

Al igual que en cualquier aplicación distribuida, las aplicaciones orientadas a servicios son aplicaciones multicapa y tienen capas de presentación, lógica de negocio y persistencia.



Las dos capas clave en una aplicación orientada a servicios son la de servicios y la de la lógica de negocio.

CAPA DE SERVICIOS

Como ya hemos dicho, los servicios son los bloques de construcción de las aplicaciones orientadas a servicios. Los servicios con auto-contenidos, mantienen su propio estado, y proporcionan una interfaz con bajo acoplamiento.

El mayor reto cuando construimos una aplicación orientada a servicios es crear una interfaz con el nivel adecuado de abstracción. Cuando analizamos los requerimientos del negocio, tenemos que considerar cuidadosamente qué componentes software queremos construir como servicios. Generalmente, los servicios deberían proporcionar una funcionalidad con un alto nivel de abstracción (coarse-grained). Por ejemplo, un componente software que procesa una orden de compra es un buen candidato para ser publicado como un servicio, en contraposición con un componente que solamente actualiza un atributo de una orden de compra.

El aspecto más importante de un servicio es su descripción. Cuando utilizamos servicios Web como tecnología de implementación para una SOA, el Web Service Description Language (WSDL) describe los mensajes, tipos y operaciones del servicio Web, que constituyen el contrato de dichos servicios.

CAPA LÓGICA DE NEGOCIO

Otra "promesa" de SOA es que podemos construir nuevas aplicaciones a partir de servicios existentes. El principal beneficio que proporciona SOA es la estandarización del modelado de procesos de negocio, a menudo referido como orquestación de servicios. Podemos construir una capa de abstracción basada en servicios Web sobre sistemas legacy y posteriormente beneficiarnos de este hecho para ensamblar procesos de negocio.

Adicionalmente, los vendedores de plataformas SOA proporcionan herramientas y servidores para diseñar y ejecutar estos procesos de negocio. Este esfuerzo se ha materializado en un estándar en OASIS denominado Business Process Execution Language (BPEL); la mayoría de vendedores de plataformas se están adhiriendo a este estándar. BPEL es esencialmente un lenguaje de programación, pero su representación es XML.

EL GOBIERNO SOA

La definición de la palabra "gobierno" (governance) implica la acción o forma de gobernar. Dentro del contexto de las IT (Information Technology), además, representa un marco de

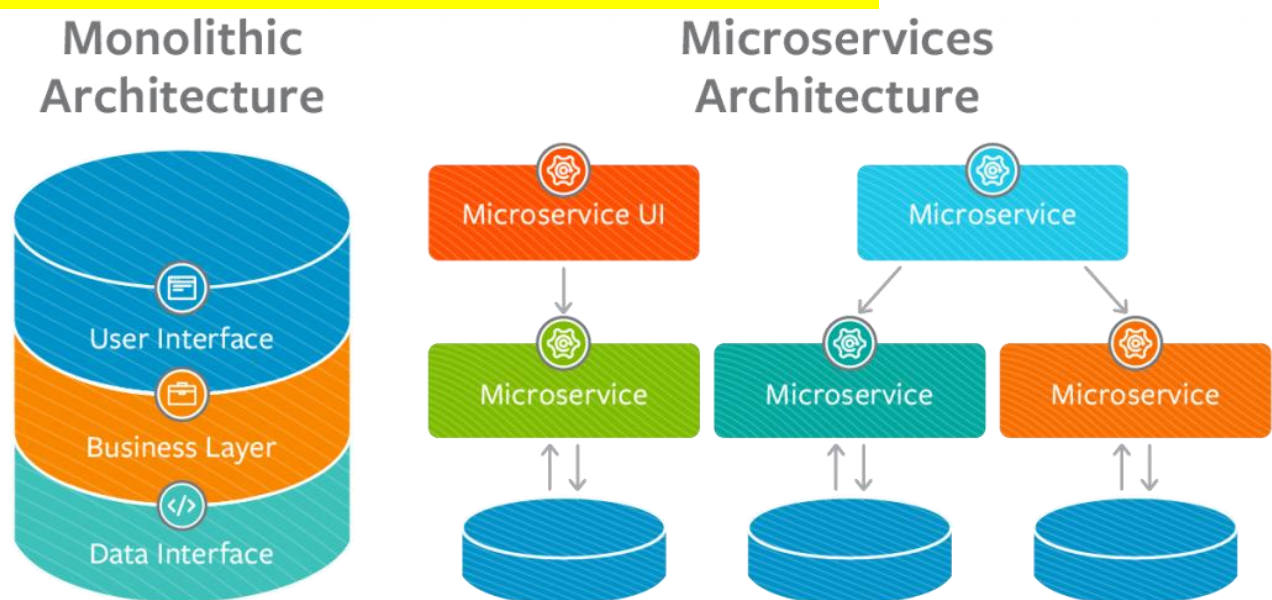
toma de decisiones y responsabilidades que fomenta un comportamiento deseable en las IT.

Los participantes en el gobierno deciden políticas sobre diferentes categorías de decisiones que deben realizarse. Dichos participantes también llevan a cabo la identificación de roles entre la gente que conforma la empresa. Los miembros del grupo de gobierno también identifican a los expertos en diferentes materias de quienes se espera que proporcionen las "entradas" para acordar decisiones, así como identifican al grupo de gente que debe tener ejercer sus responsabilidades (basadas en sus roles). Un equipo de gobierno de IT debe tratar tres cuestiones:

- Decisiones que se deben tomar para asegurar una gestión y uso efectivos de la IT.
- Autoridades que deben tomar dichas decisiones.
- Manera en cómo definir dichas decisiones y cómo pueden monitorizarse.



Tema 2: INTRODUCCIÓN A MICROSERVICIOS



Arquitectura de Software

La arquitectura de software define guías generales que indican la estructura, funcionamiento e interacción, entre las partes del software.

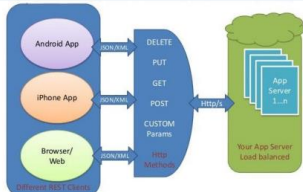
Estilo arquitectónico

- Un estilo arquitectónico define una familia de sistemas (cierto tipo de sistemas) en términos de patrones estructurales, de control, de comunicación.
- Los estilos de arquitectura no requieren el uso de tecnologías concretas, pero algunas tecnologías son adecuadas para ciertas arquitecturas.

El estilo arquitectónico, describe:

- Un conjunto de componentes.
- Un conjunto de conectores entre componentes (comunicación, coordinación, cooperación, etc.).
- Restricciones que definen cómo se integran los componentes para formar el sistema.
- Modelos que permiten comprender las propiedades de un sistema general.

ESTILOS DE ARQUITECTURA

Estilo de arquitectura	Descripción	Diagrama
Orientada al Servicio (SOA)	Se forma la estructura de una aplicación descomponiéndola en varios servicios (normalmente como servicios HTTP) que se comunican por medio de un servicio de bus de mensajes.	
Representational State Transfer (REST)	Se refiere a una interfaz, que consiste en una red de recursos Web relacionada por links y operaciones como GET, DELETE (transiciones de estado).	
Microservicios	Consta de una colección de servicios autónomos y pequeños. Los servicios son independientes entre sí y cada uno debe implementar una funcionalidad de negocio individual. Los servicios están acoplados de forma flexible y se comunican a través de contratos de API.	

SERVICIO WEB

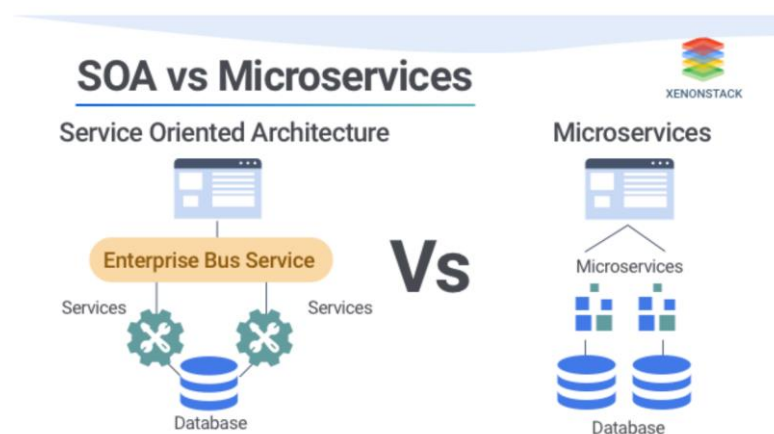
Un servicio web es un sistema software diseñado para soportar la interacción máquina-a-máquina, a través de una red, de forma interoperable. Cuenta con una interfaz descrita en un formato procesable por un equipo informático (específicamente en WSDL), a través de la que es posible interactuar con él mismo, mediante el intercambio de mensajes SOAP, típicamente transmitidos usando serialización XML sobre HTTP conjuntamente con otros estándares web.

MICROSERVICIOS

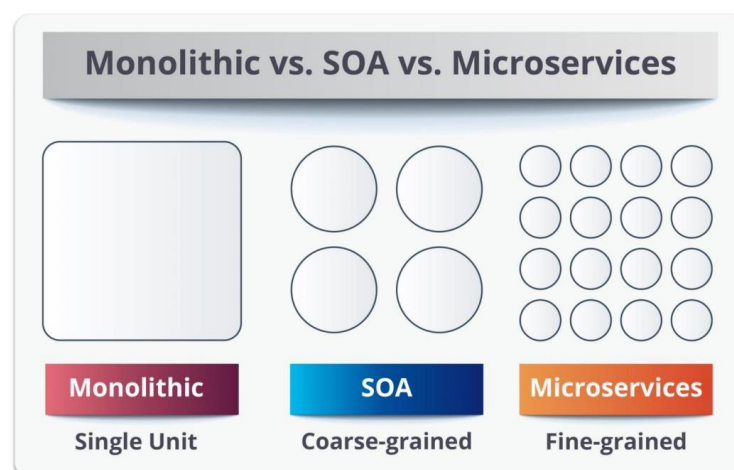
Los microservicios son una evolución de los conceptos SOA.

- Aumentan la agilidad y velocidad para hacer cambios, y la habilidad de hacerlos con un costo total e impacto menores en la infraestructura existente.
- Aunque los microservicios se derivan de SOA, no es lo mismo que la arquitectura de microservicios.

REPRESENTACIÓN GRÁFICA SOA VS MICROSERVICIOS



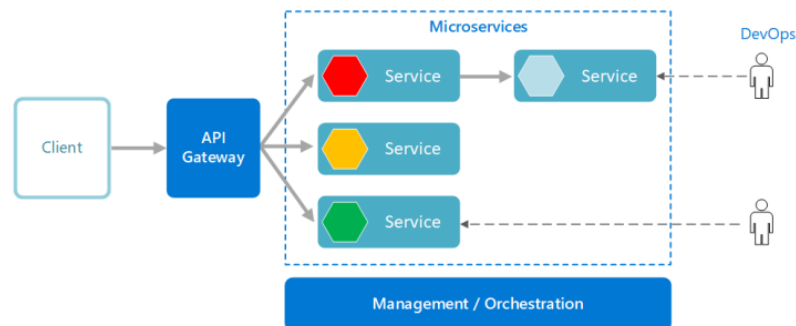
MONOLÍTICA, SOA Y MICROSERVICIOS



ARQUITECTURA DE MICROSERVICIOS

Utilizada para crear aplicaciones usando un conjunto de pequeños servicios, los cuales se comunican entre sí, pero se ejecutan de forma individual.

- Dr. Peter Rodgers (investigador en HP) en 2005 uso el término “Micro Web Services”.
- En 2011 se usó por primera vez el concepto “microservicios”.

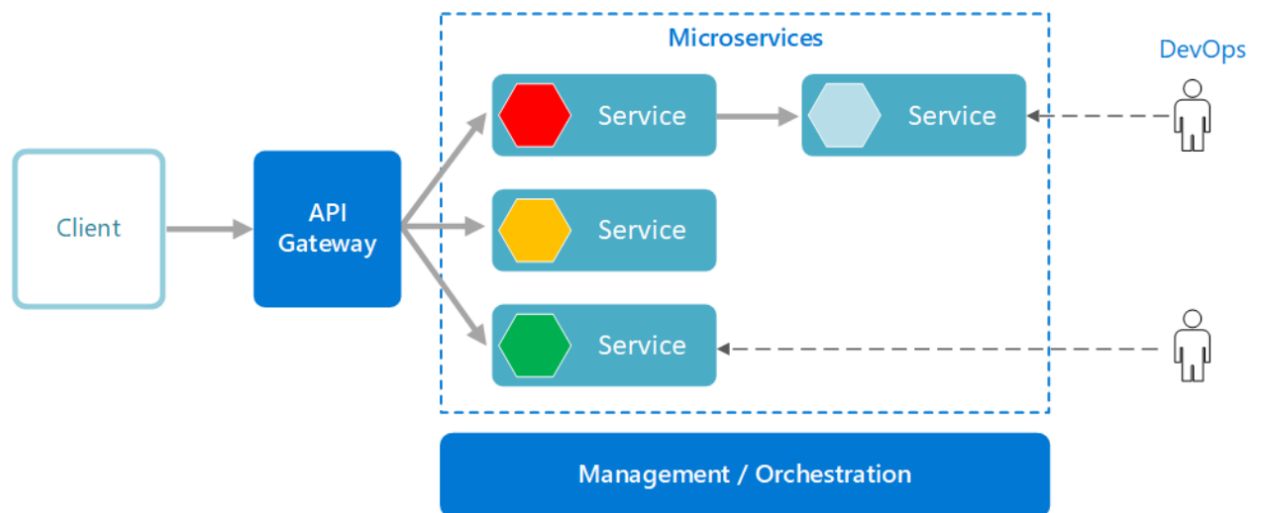


CARACTERÍSTICAS DE LOS MICROSERVICIOS

Son pequeños e independientes.	Cada servicio puede administrarse por un equipo de desarrollo pequeño.	Los servicios pueden actualizarse sin tener que volver a implementar toda la aplicación.
Los servicios son los responsables de conservar sus propios datos o estado externo.	Los servicios se comunican mediante API bien definidas.	No es necesario que los servicios compartan la misma pila de tecnología, las bibliotecas o los marcos de trabajo.

ELEMENTOS DE UNA ARQUITECTURA DE MICROSERVICIOS

- Puerta de enlace de API.
- Microservicio.
- Consumidores o clientes.
- Administración u orquestación.



Puerta de enlace de API

Es el punto de entrada para los clientes. En lugar de llamar a los servicios directamente.

- La mayor ventaja de usar una puerta de enlace de API es que desacopla los clientes de los servicios.
- Los servicios pueden cambiar de versión o refactorizarse sin necesidad de actualizar todos los clientes.



Sabías que. - La puerta de enlace de API es una de las principales características de los microservicios.

REFORZANDO LA DEFINICIÓN DE MICROSERVICIOS

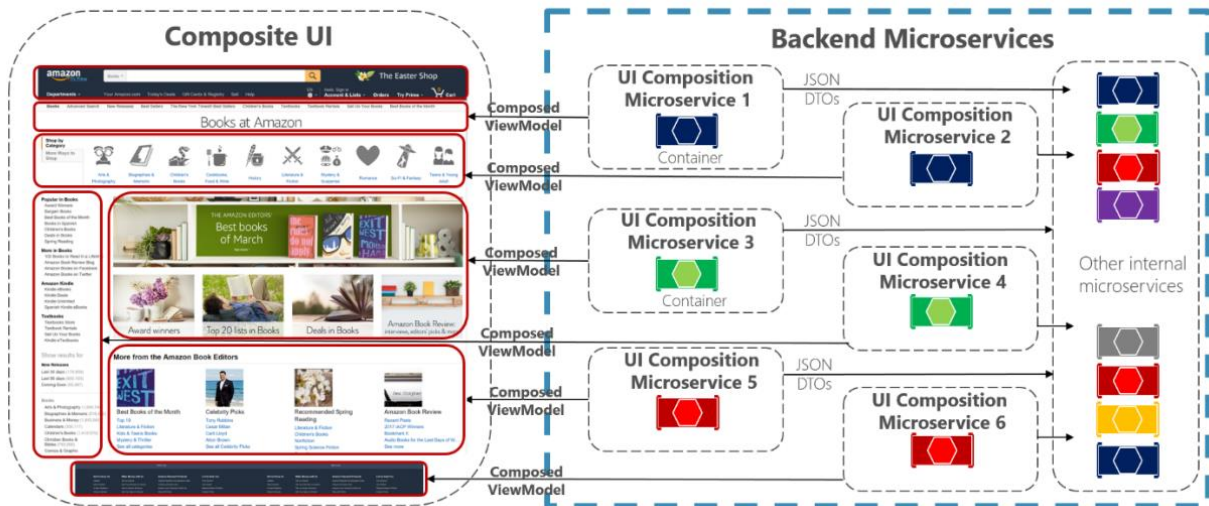
Es una función bien definida, pequeña, autocontenida e independiente del contexto o estado de otros servicios cuyas características son la reutilización, estandarización, abstracción y autonomía.

CONSUMIDORES O CLIENTES

- Puede ser una aplicación móvil, sistema, módulo de software u otro servicio.
- Demanda la funcionalidad específica que el servicio proporciona.
- Se ejecuta en una interfaz definida llamando a los microservicios como recursos.

INTERFAZ DISEÑADA PARA MICROSERVICIOS

Composite UI *generated* by microservices



VENTAJAS DE LOS MICROSERVICIOS

- Agilidad.
- Equipos pequeños y centrados.
- Base de código pequeña.
- Mezcla de tecnologías.
- Aislamiento de errores.
- Escalabilidad.
- Aislamiento de los datos.

DESVENTAJAS DE LOS MICROSERVICIOS

- Complejidad.
- Desarrollo y pruebas.
- Falta de gobernanza.
- Congestión y latencia de red.
- Dificultad para la integridad de datos.
- Administración compleja.
- Control de versiones delicada.
- Evaluar conjunto de habilidades del equipo.



Enlace para complementar el conocimiento sobre microservicios.

MICROSERVICIOS → <https://www.uv.mx/personal/ermeneses/files/2020/09/Clase16-Intro-a-Microservicios.pdf>



Tema 3: FRAMEWORKS

En pocas palabras, un Framework es una estructura previa que se puede aprovechar para desarrollar un proyecto.

- El Framework es una especie de plantilla, un esquema conceptual, que simplifica la elaboración de una tarea, ya que solo es necesario complementarlo de acuerdo a lo que se quiere realizar.
- A pesar de que su uso más común es en la informática, este concepto es también utilizado en el Marketing.

STACK PARA MICROSERVICIOS

➤ **NodeJS**

- Express + Baucis.
- SenecaJS.
- FeatherJS.
- Serverless: Claudia.



➤ **Java**

- Jersey.
- Dropwizard.
- Sprint Boot.
- Play.
- SparkJava.



➤ **.NET Core**

- NancyFX.
- WebAPI.



➤ **Go**

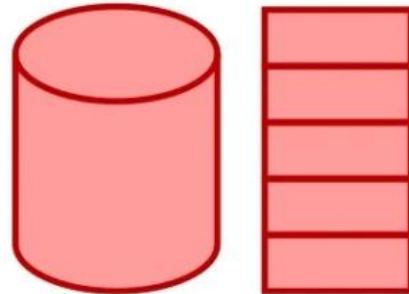
- go - kit/kit.
- Micro/go - micro.
- goa.design.
- coding/kite.



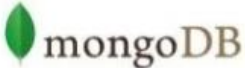
- Serverless: Sparta, Apex, Gordon.

PERSISTENCIA – BASES DE DATOS

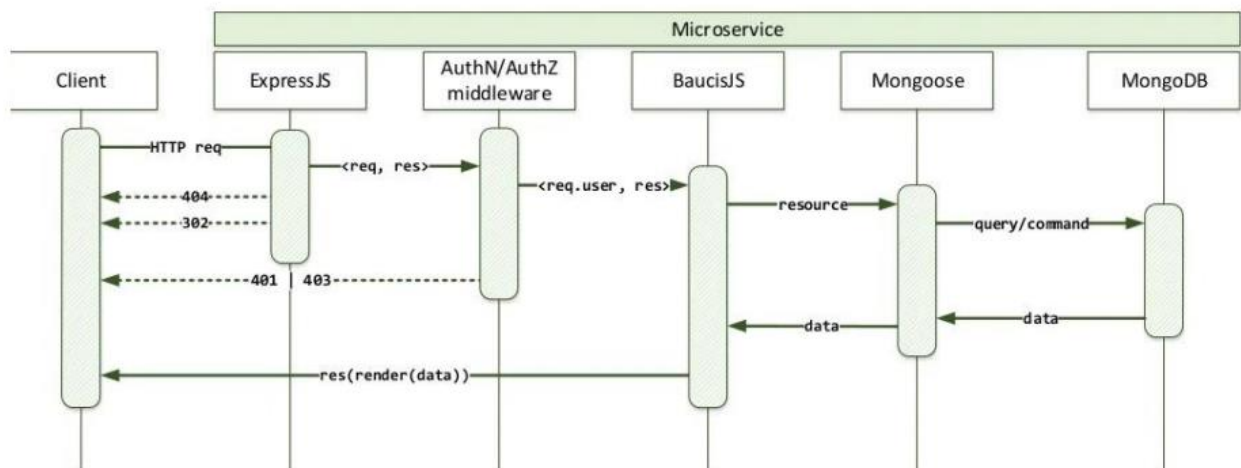
- NoSQL
 - MongoDB, CouchDB, Dynamo, DocumentDB
- RDBM
 - MySQL, Posgress, Oracle, Sql Server
- Key-Value Stores
 - Redis
 - Memcache
- Colas
 - RabbitMQ, ZeroMQ, AMZ SQS, Azure Queues
- Blobs
 - Amazon S3
 - Azure Blobs



STACK MEAN - PUERTOS

	Dev	Nube	Producción
  JS	Local :27001	db :27001	cluster :27001
  JS	Local :5000	app :80	app :80
  JS	-	-	- lb: 443
  JS	Navegador	Navegador	Navegador

ARQUITECTURA DE STACK MEAN



EJEMPLO DE MICROSERVICIO CON EXPRESS DE NODEJS



```
var express = require('express');
var app = express();

app.get('/hello', function(req, res) {
  res.status(200).send('hello world');
});
```

EJEMPLO DE MICROSERVICIO CON NANCYFX DE .NET CORE



```
public class SampleModule : Nancy.NancyModule
{
    public SampleModule()
    {
        Get["/hello"] = _ => "hello world";
    }
}
```

Para finalizar, se podría mencionar que dentro de la arquitectura se tiene dos componentes esenciales que deben manejar como futuros desarrolladores o arquitectos de software:

FRONTEND | BACKEND



Videos que permitirán complementar lo aprendido durante las sesiones.

SOA → <https://www.youtube.com/watch?v=ftsnZiyVF40>

MICROSERVICIOS → <https://www.youtube.com/watch?v=9R2hFwIPGnQ>

FRAMEWORKS → <https://www.youtube.com/watch?v=QBC87tMv350>

Referencias Bibliográficas

Ermeneses, P. (2020). Introducción a los microservicios. Universidad de Estadística e Informática. Recuperado desde: <https://www.uv.mx/personal/ermeneses/files/2020/09/Clase16-Intro-a-Microservicios.pdf>

Departamento de Tecnología de Tijuana (2007). Arquitectura orientada a servicios (SOA). Recuperado desde: <http://www.jtech.ua.es/j2ee/2006-2007/restringido/int/sesion02-apuntes.pdf>

Bucheli, L. (2021). Manual sobre Frameworks para Microservicios.